

```

        """Initialize the battery's attributes."""
        self.battery_size = battery_size

❷ def describe_battery(self):
    """Print a statement describing the battery size."""
    print(f"This car has a {self.battery_size}-kWh battery.")

class ElectricCar(Car):
    """Represent aspects of a car, specific to electric vehicles."""

    def __init__(self, make, model, year):
        """
        Initialize attributes of the parent class.
        Then initialize attributes specific to an electric car.
        """
        super().__init__(make, model, year)
❸ self.battery = Battery()

my_leaf = ElectricCar('nissan', 'leaf', 2024)
print(my_leaf.get_descriptive_name())
my_leaf.battery.describe_battery()

```

We define a new class called `Battery` that doesn't inherit from any other class. The `__init__()` method ❶ has one parameter, `battery_size`, in addition to `self`. This is an optional parameter that sets the battery's size to 40 if no value is provided. The method `describe_battery()` has been moved to this class as well ❷.

In the `ElectricCar` class, we now add an attribute called `self.battery` ❸. This line tells Python to create a new instance of `Battery` (with a default size of 40, because we're not specifying a value) and assign that instance to the attribute `self.battery`. This will happen every time the `__init__()` method is called; any `ElectricCar` instance will now have a `Battery` instance created automatically.

We create an electric car and assign it to the variable `my_leaf`. When we want to describe the battery, we need to work through the car's battery attribute:

```
my_leaf.battery.describe_battery()
```

This line tells Python to look at the instance `my_leaf`, find its `battery` attribute, and call the method `describe_battery()` that's associated with the `Battery` instance assigned to the attribute.

The output is identical to what we saw previously:

```
2024 Nissan Leaf
This car has a 40-kWh battery.
```

This looks like a lot of extra work, but now we can describe the battery in as much detail as we want without cluttering the `ElectricCar` class. Let's